DCS 404: ARTIFICIAL INTELLIGENCE & MACHINE LEARNING | CAPSTONE IMPLEMENTATION GUIDE

Spam Message & Email Detection with Machine Learning & Transformers

Final-Week Comprehensive Project Guide, Architecture & Implementation Roadmap

Westcliff University - College of Technology & Engineering | Author: Sujal | Fall 2026 | Updated per update.md

Capstone Framework Overview
Important Academic Syllabus & Capstone Context:
The DCS 404 syllabus specifies a 5-8 slide final presentation synthesizing key machine learning principles learned during Weeks 1-7. Following formal professor guidance (update.md), this project expands into an applied Natural Language Processing system demonstrating data hygiene, TF-IDF vectorization, class imbalance remediation via SMOTE, traditional statistical learning, deep transformer sequence modeling (DistilBERT & DistilRoBERTa), and production deployment in Streamlit. This document provides the definitive implementation roadmap and theoretical blueprint.

1. Why This Project Fits the Course

Spam message detection represents an archetypal supervised NLP classification challenge. It directly synthesizes every major theoretical and empirical concept covered in DCS 404:

Course Topic / Week	How It Appears in This Project
Python Basics / NumPy	Load, clean, deduplicate, and transform text data; manage sparse matrix structures.
Problem Framing & Hygiene	Frame binary classification (Spam vs. Ham); establish strict anti-leakage train/test splits.
Data Prep & TF-IDF Extraction	Tokenize text, remove English stopwords, and construct 5,000 unigram/bigram TF-IDF features.
Class Imbalance (SMOTE)	Apply Synthetic Minority Over-sampling Technique exclusively to training feature representations.
Supervised Linear Classification	Train Logistic Regression with L2 regularization as the primary SMOTE-based classifier.
Probabilistic Classification	Train Multinomial Naive Bayes; demonstrate why SMOTE degrades discrete word priors.
Deep Learning & Transformers	Fine-tune DistilBERT and DistilRoBERTa sequence classification heads with PyTorch in Google Colab.

2. End-to-End Methodology & Anti-Leakage Protocol

Data leakage is the most critical methodological error in machine learning text classification. To guarantee valid, unbiased generalization performance, the project enforces a strict partitioned pipeline:

```
Raw UCI SMS Dataset (5,572 rows) -> Clean & Deduplicate (5,169 distinct rows)
    ↓
Stratified 80/20 Train/Test Split (Train: 4,135 [3,613 Ham, 522 Spam] | Test: 1,034 [903 Ham, 131
Spam])
    ↓
Fit TfidfVectorizer STRICTLY on Training Text Only (Prevent test vocabulary leakage)
    ↓
Transform Training and Test Sets using Fitted Vectorizer
    ↓
Apply SMOTE ONLY to Training Features (Synthesize minority samples -> 3,613 Ham : 3,613 Spam)
    ↓
Train Machine Learning Classifiers (Logistic Regression with SMOTE, Multinomial Naive Bayes)
    ↓
```

```
Fine-Tune Transformer Pipeline Separately (DistilBERT & DistilRoBERTa with AdamW in Colab)
    ↓
Evaluate All 4 Models on the Untouched, Authentic Held-Out Test Set (N = 1,034)
    ↓
Deploy Models to 7-Section Interactive Streamlit Dashboard (app.py)
```

Core Pipeline Principles: Methodological Rules (update.md Section 9):
1. Never fit vectorizers across the full corpus before splitting; vocabulary frequencies must not leak from test to train.
2. Never apply SMOTE or oversampling to the held-out test split. The test set must reflect true operational class prevalence (~12.6% spam).
3. Evaluate all competing models on the identical test split to ensure fair comparative benchmarking.

3. Model Architectures & Algorithmic Foundations

Logistic Regression with SMOTE (Primary Linear Model)

Logistic Regression models log-odds using the sigmoid function: P(Y=1|X) = 1 / (1 + exp(-beta^T X)). High-dimensional TF-IDF matrices (5,000 terms) are typically linearly separable. In imbalanced datasets, the linear hyperplane biases toward the majority class, producing low spam recall (78.63%) and 28 false negatives. By applying SMOTE to the training features, the decision margin is shifted, boosting spam recall to 90.08% and reducing false negatives to 13.

Multinomial Naive Bayes & The SMOTE Distortion Effect

Multinomial Naive Bayes estimates posterior class likelihoods using Bayes' Theorem with Laplace smoothing (alpha=0.2). Crucially, the professor requested an analysis of SMOTE compatibility with Naive Bayes: When SMOTE is applied, it interpolates synthetic continuous values between feature points, violating the discrete integer word frequency assumption of the multinomial distribution. Furthermore, SMOTE artificially enforces a uniform class prior P(Spam) = 0.50, causing the model to over-predict spam on the authentic test distribution. As a result, standard MNB achieves 98.26% precision with only 2 false positives, whereas applying SMOTE degrades MNB precision. This highlights a key pedagogical takeaway: geometric classifiers benefit from SMOTE, while generative Bayesian models prefer natural frequencies.

Deep Contextual Transformers: DistilBERT & DistilRoBERTa

Unlike bag-of-words representations that discard syntax and word order, transformer architectures leverage multi-head self-attention to analyze full bidirectional context. We fine-tuned two state-of-the-art distilled models: 1) DistilBERT: A 6-layer architecture retaining 97% of BERT's language understanding with 40% fewer parameters (66M) and 60% faster execution. 2) DistilRoBERTa: A distilled RoBERTa architecture (82M parameters) pre-trained with dynamic masking on larger corpora, yielding enhanced sensitivity to subtle phrasing.

4. Exploratory Data Analysis & Text Visualizations

Per Section 4 of update.md, high-resolution Word Clouds and distribution charts were generated to explore corpus patterns:

Lexical Findings: • Spam Word Cloud: Dominated by urgent calls-to-action, financial rewards, and numeric contact prompts: 'FREE', 'call', 'claim', 'prize', 'txt', 'won', 'cash', 'urgent', 'customer', 'reply', 'mobile'.

Lexical Findings: • Ham Word Cloud: Reflects everyday social communication, planning, and personal connection: 'come', 'good', 'time', 'got', 'will', 'day', 'ok', 'home', 'later', 'love', 'going', 'need'.

Structural Findings: • Length Distributions: Spam messages average 138.8 characters (median 149), closely approaching the 160 SMS limit due to legal disclaimers and promo details. Ham messages average 71.0 characters (median 52).

5. Empirical Benchmark Results (N = 1,034)

All models evaluated on the held-out test split of 1,034 messages (903 Ham, 131 Spam):

Model	Category	Accuracy	Precision	Recall	F1-Score	FP	FN
DistilBERT	Transformer	98.84%	95.42%	95.42%	95.42%	6	6
DistilRoBERTa	Transformer	98.84%	97.60%	93.13%	95.31%	3	9
Multinomial NB	Traditional ML	98.07%	98.26%	86.26%	91.87%	2	18
Logistic Reg (SMOTE)	Traditional ML	97.29%	88.72%	90.08%	89.39%	15	13

6. Multi-Dimensional Trade-Off Analysis

In accordance with Section 5 of update.md, models are compared across operational dimensions rather than naming a single 'winner':

Evaluation Metric	Traditional ML (LR / MNB)	Transformers (DistilBERT / RoBERTa)
Accuracy & F1-Score	97.29% - 98.07% Acc, 89.4% - 91.9% F1	98.84% Acc, 95.3% - 95.4% F1 (+3.5% to +6% F1 gain)
Training Execution Time	0.01 - 0.05 seconds (Instantaneous)	~2,849 seconds on CPU / ~3 min on GPU (~50,000x difference)
Inference Latency	0.0008 ms/sample (~1,250,000 msg/sec)	70.2 - 71.3 ms/sample (~14 msg/sec on CPU) (~90,000x difference)
Hardware Requirements	Minimal (Standard CPU, <50 MB RAM)	Heavy (Dedicated GPU required for training, 2+ GB RAM)
Deployment Complexity	Single lightweight joblib artifact (~230 KB)	Large PyTorch safetensors artifact (~260 MB) + runtime packages

7. Suggested Google Colab Notebook Structure

The project notebook (notebooks/training.ipynb) is organized into 12 structured cells for complete reproducibility:

• **Cell 1: Environment Setup:** Installs imbalanced-learn, wordcloud, transformers, and torch; verifies GPU acceleration.

• **Cell 2: Data Ingestion:** Downloads raw UCI SMS Spam Collection; parses tab-separated records into pandas DataFrame.

• **Cell 3: Data Cleaning:** Prunes exact duplicates (403 rows); maps labels to binary integers {ham: 0, spam: 1}.

• **Cell 4: Exploratory Analysis:** Calculates character/word counts; plots message length distributions and class frequencies.

• **Cell 5: Word Clouds:** Generates and renders high-resolution Spam and Ham Word Clouds using WordCloud library.

• **Cell 6: Train/Test Partitioning:** Executes stratified 80/20 train/test split (4,135 train, 1,034 test); verifies zero test contamination.

• **Cell 7: TF-IDF Extraction:** Fits TfidfVectorizer strictly on training text; transforms training and testing feature partitions.

• **Cell 8: SMOTE Oversampling:** Applies SMOTE to training TF-IDF features; balances train set to 3,613:3,613; shows before/after counts.

• **Cell 9: Logistic Regression:** Trains LogisticRegression on SMOTE features; computes test predictions, classification report, and ROC-AUC.

• **Cell 10: Naive Bayes Analysis:** Trains MultinomialNB with and without SMOTE; explains why SMOTE degrades discrete Bayesian likelihoods.

• **Cell 11: Transformer Fine-Tuning:** Fine-tunes DistilBERT and DistilRoBERTa on binary classification using PyTorch and Hugging Face Trainer.

• **Cell 12: Benchmark & Export:** Generates 4-model comparison table, confusion matrix grid, ROC curves, and serializes model weights.

8. Starter Code Blueprint: Anti-Leakage SMOTE Pipeline

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE

# 1. Ingestion & Cleaning
df = pd.read_csv('data/SMSSpamCollection', sep='\t', header=None, names=['label',
'text']).drop_duplicates()
df['y'] = df['label'].map({'ham': 0, 'spam': 1})

# 2. Stratified Split (BEFORE vectorization)
X_train, X_test, y_train, y_test = train_test_split(
    df['text'], df['y'], test_size=0.20, random_state=42, stratify=df['y']
)

# 3. Fit TF-IDF on Training Data Only
tfidf = TfidfVectorizer(max_features=5000, stop_words='english', ngram_range=(1,2))
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec = tfidf.transform(X_test)

# 4. Apply SMOTE to Training Features Only
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_vec, y_train)

# 5. Train Model & Evaluate on Untouched Test Set
model = LogisticRegression(C=2.0, max_iter=1000, random_state=42)
model.fit(X_train_smote, y_train_smote)
y_pred = model.predict(X_test_vec)
print(classification_report(y_test, y_pred, target_names=['Ham', 'Spam']))
```

9. Final Submission Checklist

[√] Google Colab & Jupyter notebook (notebooks/training.ipynb) executes cleanly without runtime errors.

[√] Anti-leakage protocol verified: TfidfVectorizer fitted strictly on training partition.

[√] SMOTE applied exclusively to training TF-IDF features; held-out test split remains 100% authentic.

[√] Logistic Regression trained with SMOTE; Multinomial Naive Bayes evaluated with SMOTE trade-off analysis.

[√] DistilBERT and DistilRoBERTa fine-tuned for sequence classification and benchmarked.

[√] Spam and Ham Word Clouds generated and rendered in Exploratory Data Analysis.

[√] Evaluation metrics reported: Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrices.

[√] Multi-dimensional trade-off analysis documented (accuracy, speed, complexity, hardware, deployment).

[√] Streamlit application (app.py) runs locally with 7 functional sections and interactive live prediction.

[√] Final presentation package (presentation/) includes 8-slide PPTX, PDF, and speaker notes.